

Explicacion detallada de los 5 modulos Odoo

Objetivo general

Se desarrollaron 5 modulos personalizados de Odoo 19 para representar una gestion academica basica. Los modulos estan separados para cumplir la indicacion de que no sea un solo modulo, sino cinco addons instalables. Cada modulo contiene su propio modelo, vistas basicas, accion de ventana, menu, grupos de seguridad y permisos.

Los modulos creados son:

1. `jj_academia_carrera`: administra carreras academicas.
2. `jj_academia_estudiante`: administra estudiantes y los relaciona con carreras.
3. `jj_academia_docente`: administra docentes.
4. `jj_academia_asignatura`: administra asignaturas y las relaciona con carreras y docentes.
5. `jj_academia_matricula`: administra matriculas y las relaciona con estudiantes y asignaturas.

Tambien se corrigio el archivo `odoo.conf` para que el `addons_path` apunte a la carpeta `custom-addons`, donde quedaron ubicados los cinco modulos personalizados.

Estructura general creada

Cada modulo tiene la siguiente estructura:

```
custom-addons/nombre_del_modulo/  
├── __init__.py  
├── __manifest__.py  
├── models/  
│   ├── __init__.py  
│   └── archivo_modelo.py  
├── security/  
│   ├── security.xml  
│   └── ir.model.access.csv  
└── views/  
    └── archivo_vistas.xml
```

El archivo `__manifest__.py` le dice a Odoo el nombre del modulo, la version, la categoria, las dependencias y los archivos XML/CSV que debe cargar. El archivo

`__init__.py` permite que Python importe la carpeta `models`. En `models` se escriben las clases Python que crean las tablas. En `security` se crean los grupos y permisos. En `views` se crean las pantallas visibles para el usuario.

Modulo 1: JJ Academia - Carreras

Ruta: `custom-addons/jj_academia_carrera`

Modelo creado: `jj.carrera`

Este modulo representa las carreras academicas de una institucion. Por ejemplo: Ingenieria en Software, Administracion, Contabilidad, etc.

Campos principales:

- `name`: campo Char, guarda el nombre de la carrera. Es obligatorio.
- `code`: campo Char, guarda un codigo unico de carrera.
- `active`: campo Boolean, indica si la carrera esta activa.
- `modality`: campo Selection, permite elegir entre presencial, virtual o hibrida.
- `duration_semesters`: campo Integer, guarda la duracion en semestres.
- `monthly_fee`: campo Float, guarda el valor mensual de la carrera.
- `start_date`: campo Date, guarda la fecha de inicio.
- `coordinator_id`: campo Many2one hacia `res.partner`, relaciona la carrera con un coordinador.
- `description`: campo Text, permite escribir una descripcion larga.
- `total_estimated_cost`: campo Float calculado, muestra el costo estimado total.

Decoradores usados:

- `@api.depends`: se usa en `_compute_total_estimated_cost`. Recalcula el costo total cuando cambia `duration_semesters` o `monthly_fee`.
- `@api.onchange`: se usa en `_onchange_modality`. Cuando el usuario cambia la modalidad, Odoo propone automaticamente una mensualidad.
- `@api.constrains`: se usa en `_check_academic_values`. Valida que la duracion sea mayor que cero, que la mensualidad no sea negativa y que el codigo tenga al menos 3 caracteres.

Regla SQL:

- `code_unique`: impide repetir el codigo de una carrera.

Vistas:

- Vista list: muestra carreras en forma de tabla.
- Vista form: muestra el formulario completo de una carrera.

Accion y menu:

- Accion: action_jj_carrera.
- Menu principal: JJ Carreras.
- Submenu: Carreras.

Seguridad:

- Grupo Usuario: puede leer carreras.
- Grupo Administrador: puede leer, crear, editar y eliminar carreras.

Modulo 2: JJ Academia - Estudiantes

Ruta: custom-addons/jj_academia_estudiante

Modelo creado: jj.estudiante

Este modulo administra los estudiantes. Depende del modulo de carreras porque cada estudiante pertenece a una carrera.

Campos principales:

- name: campo Char, nombre completo del estudiante.
- document: campo Char, cedula o documento unico.
- age: campo Integer, edad del estudiante.
- email: campo Char, correo electronico.
- phone: campo Char, telefono.
- active: campo Boolean, indica si el estudiante esta activo.
- enrollment_date: campo Date, fecha de ingreso.
- career_id: campo Many2one hacia jj.carrera, relaciona estudiante con carrera.
- semester: campo Selection, semestre actual.
- average: campo Float, promedio academico.
- scholarship: campo Boolean, indica si tiene beca.
- scholarship_percent: campo Float, porcentaje de beca.
- final_monthly_fee: campo Float calculado, mensualidad con descuento.
- status: campo Selection, estado del estudiante.

- notes: campo Text, observaciones.

Decoradores usados:

- `@api.depends`: recalcula `final_monthly_fee` cuando cambia la mensualidad de la carrera, si tiene beca o el porcentaje de beca.
- `@api.onchange`: cuando se marca o desmarca `scholarship`, Odoo coloca un porcentaje sugerido o lo deja en cero.
- `@api.constrains`: valida edad minima, promedio entre 0 y 10, y beca entre 0% y 100%.

Regla SQL:

- `document_unique`: evita que dos estudiantes tengan el mismo documento.

Vistas:

- Vista `list`: resume estudiante, carrera, semestre, promedio y beca.
- Vista `form`: muestra datos personales, academicos y observaciones.

Accion y menu:

- Accion: `action_jj_estudiante`.
- Menu principal: JJ Estudiantes.
- Submenu: Estudiantes.

Seguridad:

- Grupo Usuario: solo lectura.
- Grupo Administrador: permisos completos.

Modulo 3: JJ Academia - Docentes

Ruta: `custom-addons/jj_academia_docente`

Modelo creado: `jj.docente`

Este modulo administra docentes o profesores.

Campos principales:

- `name`: campo Char, nombre completo.
- `identification`: campo Char, identificacion unica.
- `email`: campo Char, correo.
- `phone`: campo Char, telefono.

- `active`: campo Boolean, estado activo/inactivo.
- `hire_date`: campo Date, fecha de contratacion.
- `contract_type`: campo Selection, tipo de contrato.
- `hourly_rate`: campo Float, valor por hora.
- `weekly_hours`: campo Integer, horas semanales.
- `monthly_salary`: campo Float calculado, salario mensual estimado.
- `has_master_degree`: campo Boolean, indica si tiene maestria.
- `specialty`: campo Char, especialidad.
- `biography`: campo Text, biografia.

Decoradores usados:

- `@api.depends`: calcula `monthly_salary` multiplicando valor por hora, horas semanales y 4 semanas.
- `@api.onchange`: al cambiar el tipo de contrato, asigna horas y tarifa sugeridas.
- `@api.constrains`: valida que el valor por hora sea positivo y que las horas semanales esten entre 1 y 40.

Regla SQL:

- `identification_unique`: evita identificaciones repetidas.

Vistas:

- Vista `list`: lista docentes con contrato, especialidad, horas y salario.
- Vista `form`: formulario con informacion personal, laboral y biografia.

Accion y menu:

- Accion: `action_jj_docente`.
- Menu principal: JJ Docentes.
- Submenu: Docentes.

Seguridad:

- Grupo Usuario: solo lectura.
- Grupo Administrador: permisos completos.

Modulo 4: JJ Academia - Asignaturas

Ruta: `custom-addons/jj_academia_asignatura`

Modelo creado: `jj.asignatura`

Este modulo administra asignaturas. Depende de carreras y docentes porque una asignatura pertenece a una carrera y puede tener un docente asignado.

Campos principales:

- `name`: campo Char, nombre de la asignatura.
- `code`: campo Char, codigo unico.
- `career_id`: campo Many2one hacia `jj.carrera`.
- `teacher_id`: campo Many2one hacia `jj.docente`.
- `credits`: campo Integer, numero de creditos.
- `weekly_hours`: campo Integer, horas semanales.
- `classroom_capacity`: campo Integer, cupo.
- `level`: campo Selection, nivel basico, intermedio o avanzado.
- `is_virtual`: campo Boolean, indica si es virtual.
- `teacher_hourly_rate`: campo Float relacionado, toma el valor por hora del docente.
- `estimated_monthly_cost`: campo Float calculado.
- `description`: campo Text, descripcion.

Decoradores usados:

- `@api.depends`: calcula el costo mensual estimado usando horas semanales y tarifa del docente.
- `@api.onchange`: si la asignatura es virtual, sube el cupo a 80; si no es virtual, sugiere 30.
- `@api.constrains`: valida creditos, horas y cupo mayores que cero.

Regla SQL:

- `code_unique`: evita codigos de asignatura repetidos.

Vistas:

- Vista `list`: muestra asignatura, carrera, docente, creditos, cupo y costo.
- Vista `form`: formulario para configurar la asignatura.

Accion y menu:

- Accion: `action_jj_asignatura`.
- Menu principal: JJ Asignaturas.

- Submenu: Asignaturas.

Seguridad:

- Grupo Usuario: solo lectura.
- Grupo Administrador: permisos completos.

Modulo 5: JJ Academia - Matriculas

Ruta: custom-addons/jj_academia_matricula

Modelo creado: jj.matricula

Este modulo administra matriculas academicas. Depende de estudiantes y asignaturas porque una matricula une a un estudiante con una asignatura.

Campos principales:

- name: campo Char, referencia de la matricula.
- student_id: campo Many2one hacia jj.estudiante.
- subject_id: campo Many2one hacia jj.asignatura.
- registration_date: campo Date, fecha de matricula.
- period: campo Selection, periodo academico.
- amount: campo Float, valor de matricula.
- paid: campo Boolean, indica si esta pagado.
- first_grade: campo Float, primera nota.
- second_grade: campo Float, segunda nota.
- final_grade: campo Float calculado, promedio de las dos notas.
- approved: campo Boolean calculado, indica si aprueba con nota final mayor o igual a 7.
- state: campo Selection, borrador, confirmada o cancelada.
- observations: campo Text, observaciones.

Decoradores usados:

- @api.depends: calcula final_grade y approved cuando cambian las notas.
- @api.onchange: al seleccionar una asignatura, calcula el valor de matricula segun los creditos y crea una referencia con estudiante y asignatura.
- @api.constrains: valida notas entre 0 y 10 y que el valor de matricula no sea negativo.

Vistas:

- Vista list: muestra matriculas en tabla.
- Vista form: permite registrar estudiante, asignatura, periodo, pago y notas.

Accion y menu:

- Accion: action_jj_matricula.
- Menu principal: JJ Matriculas.
- Submenu: Matriculas.

Seguridad:

- Grupo Usuario: solo lectura.
- Grupo Administrador: permisos completos.

Relaciones entre modulos

Las relaciones principales son:

- `jj.estudiante.career_id` conecta estudiantes con carreras.
- `jj.asignatura.career_id` conecta asignaturas con carreras.
- `jj.asignatura.teacher_id` conecta asignaturas con docentes.
- `jj.matricula.student_id` conecta matriculas con estudiantes.
- `jj.matricula.subject_id` conecta matriculas con asignaturas.

Estas relaciones permiten demostrar el uso de campos relacionales Many2One entre modelos personalizados.

Permisos y grupos

Cada modulo tiene dos grupos:

- Usuario: puede consultar registros, pero no crear, editar ni eliminar.
- Administrador: puede consultar, crear, editar y eliminar.

Esto se implemento con:

- `security/security.xml`: crea los grupos.
- `security/ir.model.access.csv`: asigna permisos por modelo.

Ejemplo de permisos:

```
perm_read,perm_write,perm_create,perm_unlink
1,0,0,0  -> Usuario solo lectura
1,1,1,1  -> Administrador acceso total
```


Vistas, acciones y menus

Cada modulo incluye:

- Una vista `list` para ver varios registros.
- Una vista `form` para crear o editar un registro.
- Una accion `ir.actions.act_window` para abrir el modelo.
- Un menu `ir.ui.menu` para acceder desde la interfaz de Odoo.

Con esto se cumple el requisito de vistas basicas, accion y menu por cada modelo.

Archivos modificados o creados

- `odoo.conf`: se corrigio `addons_path`.
- `custom-addons/jj_academia_carrera`: modulo de carreras.
- `custom-addons/jj_academia_estudiante`: modulo de estudiantes.
- `custom-addons/jj_academia_docente`: modulo de docentes.
- `custom-addons/jj_academia_asignatura`: modulo de asignaturas.
- `custom-addons/jj_academia_matricula`: modulo de matriculas.
- `docs/explicacion_modulos_odoo.md`: este documento explicativo.

Recomendacion para el PDF

Para la entrega, este archivo Markdown puede convertirse a PDF y al final se debe pegar la captura del ultimo cambio subido al repositorio de GitHub solicitado por el docente.

Anexo final: captura de GitHub

En esta seccion se debe adjuntar la captura del ultimo cambio subido al repositorio:

https://github.com/czambrano1997/ProgII_UTE202601

La captura debe mostrar el commit o la actividad mas reciente donde aparezcan los cambios de estos 5 modulos. Esta parte se completa despues de hacer el `git add`, `git commit` y `git push` correspondientes.